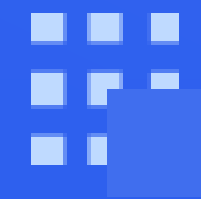


03

集成测试



集成测试概述

什么是集成测试？

集成测试是单元测试的延伸，用于验证软件组件集成后的功能。它确保组件间协作正常，并发现集成过程中出现的缺陷。集成测试旨在验证软件组件的协同工作能力，并检测系统功能性和性能问题。

自动化API测试与手工API测试区别

- 确保组件集成正确。
- 检测 and 解决接口问题何依赖关系问题。
- 发现修复缺陷。

思考：

- 软件模块之间发生冲突的原因有哪些？



集成测试缺陷原因

集成测试产生缺陷的主要原因包括：

- 代码逻辑不一致：代码由不同的程序员编写的，他们的逻辑和开发方法彼此不同。
- 需求变化：客户需求变化可能导致模块代码逻辑变更，影响整个应用程序。
- 错误数据：数据格式错误可能导致跨模块传输失败，引发错误。
- 第三方服务和 API 集成：数据传输变化可能导致 API 和第三方服务接收到错误输入，产生错误响应。
- 异常处理不充分：开发人员可能无法预见所有异常场景，直到模块集成。
- 外部硬件接口：当软件与硬件不兼容时，也可能会出现错误。

集成测试缺陷示例-规格问题

规格问题：模块集成时可能因数据类型、单位等不一致导致规格问题。

对于成绩管理系统，为便于数据输入，其输入模块可接收不同类型输入并默认保存为字符型，其统计模块为便于计算将数据默认为数值型。因此，在数据对接时可能存在数据不一致问题。同样的情况还可能发生计费系统中。例如，称重模块默认单位为克，计费模块默认单位为公斤。当称重结果传入到计费模块后，计费结果可能出现错误

集成测试缺陷示例-接口缺陷与数据丢失

接口缺陷：单元测试关注模块内部具体实现，往往会忽略模块间接口调用相关缺陷。

软件包含程序模块 A 和 B，其中 A 包含三个参数 str1、str2 和 str3，功能是将 str1 所包含的 str2 去除后保存到 str3 中。B 在调用 A 误将 str1 和 str2 的位置写反，导致处理结果不正确。

数据丢失：各个模块需要协同工作，按照一致的节奏发送、接收、处理数据，否则会导致数据阻塞、丢失等。

软件包含程序模块 A 和 B：A 负责发送数据，B 负责接收、处理数据。当 A 发送数据的速度远高于 B 处理数据的速度时，可能会造成数据阻塞或数据丢失，并大大延长系统整体的工作时间。

集成测试缺陷示例-并发问题

并发问题：并发程序在集成测试时易因模块运行次序不当出现同步错误、死锁等问题。

程序模块 A 和 B 运行时均需利用资源 X 和 Y。在单元测试时，A 和 B 均可占用资源 X 和 Y，程序可正常运行。然而在集成测试时，A 和 B 可同步运行。假设 A 已得到 X，正在等待申请 Y；B 已得到 Y，正在等待申请 X。此时，由于 A 和 B 均在等待对方已占有的资源，两者陷入死锁，导致程序无法正常运行。

集成测试流程

集成测试前的准备

再集成测试前应确保所有模块通过单元测试，以避免在集成测试中混淆模块内部和模块间交互的错误，影响测试效果并增加修复成本。

集成测试流程

- 计划阶段：根据概要设计文档、项目时间表、需求规格说明书制定集成测试计划。
- 设计阶段：基于概要设计和需求规格说明书，设计集成测试方案。
- 开发阶段：配置测试环境，创建测试脚本和测试用例。
- 运行阶段：执行测试用例，记录软件运行状态和结果。
- 分析和评估阶段：分析测试日志，评估测试结果，调整测试计划。

集成测试策略概述

并行单元测试

在软件开发中，集成测试在单元测试之后进行，但由于单元测试可能耗时较长，可以采取并行方式，对已完成单元测试的模块先进行集成测试。这种模式适用于瀑布开发环境。而在敏捷开发中，每次代码提交都会触发构建和集成测试。

集成测试实践方法

1. 分析模块间的关联和依赖关系。
2. 制定合适的集成测试策略，如一次性集成或增量式集成。
3. 考虑项目实际情况选择集成测试策略。
4. 根据被测对象特点和工程环境选择集成测试策略。

一次性集成概述

什么是一次性集成？

一次性集成是指所有程序模块完成单元测试后，直接组装进行整体测试的过程。一次性集成减少了集成次数和测试工作量，无需驱动和测试桩模块，但缺点是延长了测试和错误发现时间，且难以定位缺陷和并行测试。适用于小型、结构良好、修改少或基于可信赖构件的软件系统。

示例

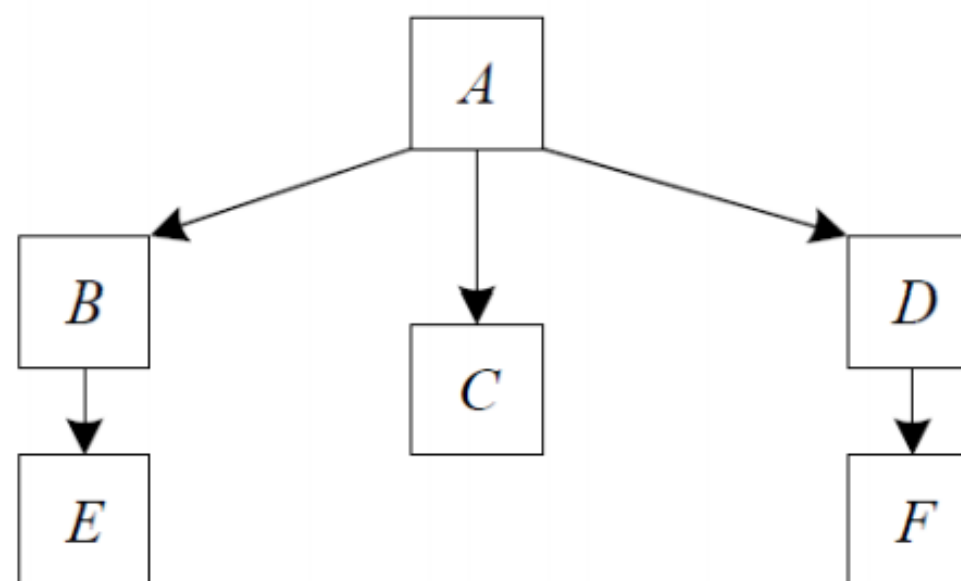


图 6.1: 软件结构示意图

●

增量式集成概述

什么是增量式集成？

增量式集成是逐步将程序模块组装测试的过程，与一次性集成相比，它能更早、更充分地发现错误并定位缺陷，但时间和工作量更大，需要编写驱动和测试桩模块。适用于大规模、复杂、并发及采用增量或框架开发的软件系统。在实践中，增量式集成比一次性集成更常见，包括自顶向下和自底向上两种典型方法。

一次性集成与增量式集对比

表 6.4: 一次性集成和增量式集成对比

类别	一次性集成	增量式集成
集成次数	少, 仅需 1 次	多
集成工作量	小	大
测试用例	少	多
驱动模块和桩模块	不需要	需要
发现错误时间	较晚	早
错误定位	难	较容易
测试程度	不彻底	较为彻底
并行性	差	较好

自顶向下集成

什么是自顶向下集成？

自顶向下集成是指依据程序结构图，从顶层开始由上到下逐步增加集成模块到集成测试的过程。在集成路径的选择上，还可选择广度优先和深度优先方法来添加测试模块。

自顶向下集成的具体步骤

1. 从根节点开始，构建测试驱动模块。
2. 添加通过单元测试的同级或下级模块，构建测试桩模块。
3. 测试并验证测试对象。
4. 重复步骤2-3，直至包含所有模块。

以深度优先为例，自顶向下增量式集成的过程如图6.2所示。其中，阴影部分为每一轮集成结果。

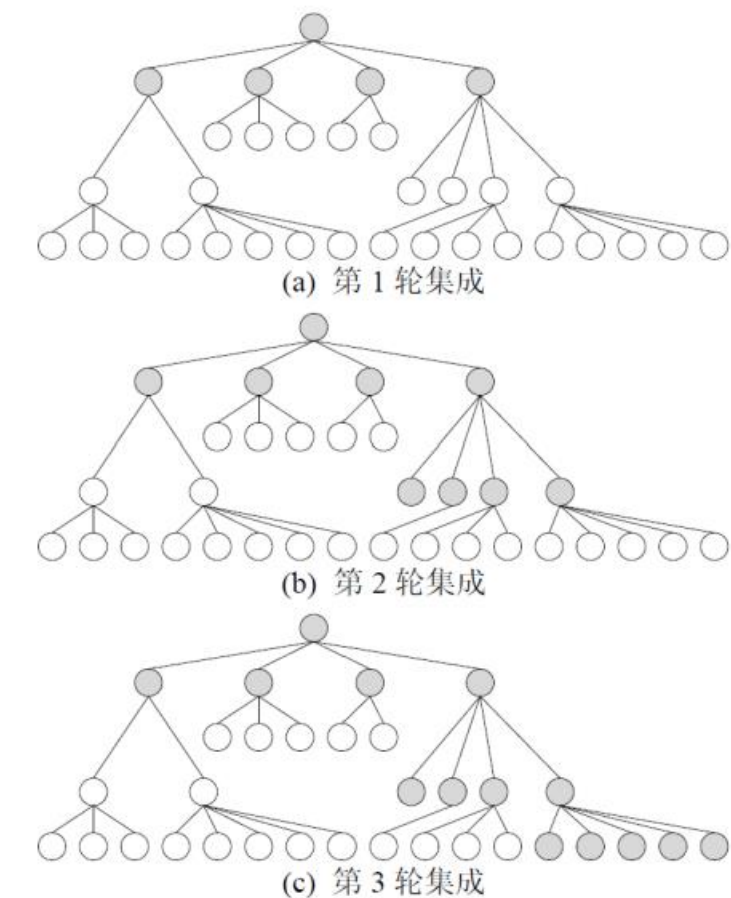


图 6.2: 深度优先自顶向下增量式集成示例

●

自顶向下集成

什么是自顶向上集成？

自顶向上集成是一种集成测试方法，它从软件结构图的顶层开始，逐步向下增加模块进行集成测试。

自顶向下集成与自底向上集成的优缺点对比

表 6.5: 自顶向下集成与自底向上集成对比

类别	自顶向下集成	自底向上集成
优点	减轻了驱动模块开发的工作量； 测试者可尽早了解系统的框架； 可以自然做到逐步求精； 若底层接口未定义或可能修改，则可以避免提交不稳定的接口。	底层叶节点的测试和集成可并行开展； 底层模块的调用和测试较为充分； 减轻了测试桩模块开发的工作量； 测试者可以较好地锁定缺陷位置； 由于驱动模块模拟了所有的调用参数，即使数据流并未构成有向的非环状图，生成测试数据也没有困难； 适合于关键模块在结构图底部的情况。
缺点	测试桩模块的开发代价较大； 底层模块无法预料的条件要求会迫使上层模块的修改； 底层模块的调用和测试不够充分； 在输入/输出模块接入系统之前，生成测试数据有一定困难； 测试桩模块不能自动生成数据，若模块间的数据流不能构成有向的非环状图，一些模块的测试数据难于生成； 观察和解释测试输出比较困难。	需要驱动模块； 高层模块的可操作性和互操作性测试不够充分； 在某些开发模式（如 XP 开发模式）上并不适用； 测试者不能尽早了解系统的框架； 时序、资源竞争等问题只有到测试后期才能被发现。

三明治式集成

什么是自顶向上集成？

三明治集成结合了自顶向下和自底向上集成，从软件结构图的中间层开始，同时向上和向下扩展。这种方法允许并行测试，更早发现和修复错误，减少对测试桩模块的需求，特别适用于具有清晰中间层的大型复杂系统。

三明治式集成示例

阴影部分表示集成过程中一个中间结果。通过三明治式集成，可以减少部分驱动模块和测试桩模块开发工作。

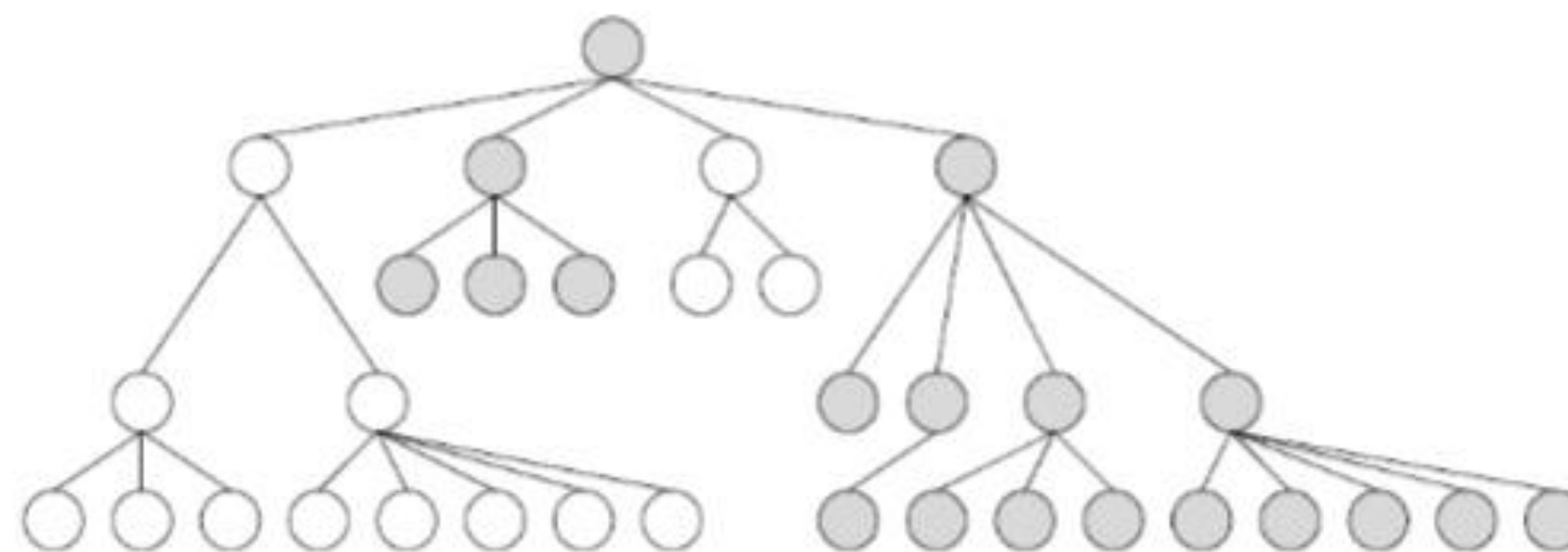


图 6.4: 三明治式集成示例

● 基于调用图的集成

什么是基于调用图的集成？

基于调用图的集成就是一种用于减少驱动模块和测试桩模块的集成策略。在基于调用图的集成策略中，需预先构建程序调用图，识别各个模块间的程序调用关系。基于调用图的集成策略主要包括成对集成和相邻集成两类。

基于调用图的集成作用

基于调用图的集成测试通过分析调用和协作关系，对成对或相邻节点进行测试，减少了驱动和测试桩模块的开发工作，充分测试了模块调用关系和功能衔接。由于每条边独立，可同时进行多组测试。然而，随着软件结构的复杂性增加，测试难度也随之增加。这种方法适用于需要快速验证调用或协作关系，以及已明确定义构件调用和协作关系的系统。

成对集成

什么是成对集成？

成对集成是指把程序调用图中的节点对放在一起进行测试，也可理解为以程序调用图中的边为单位进行测试。

成对集成示例

该图表示一个程序调用图，其中阴影部分表示成对集成过程中的一个中间结果，该中间结果包含两个节点对 $\langle 2, 7 \rangle$ 和 $\langle 4, 5 \rangle$ 。

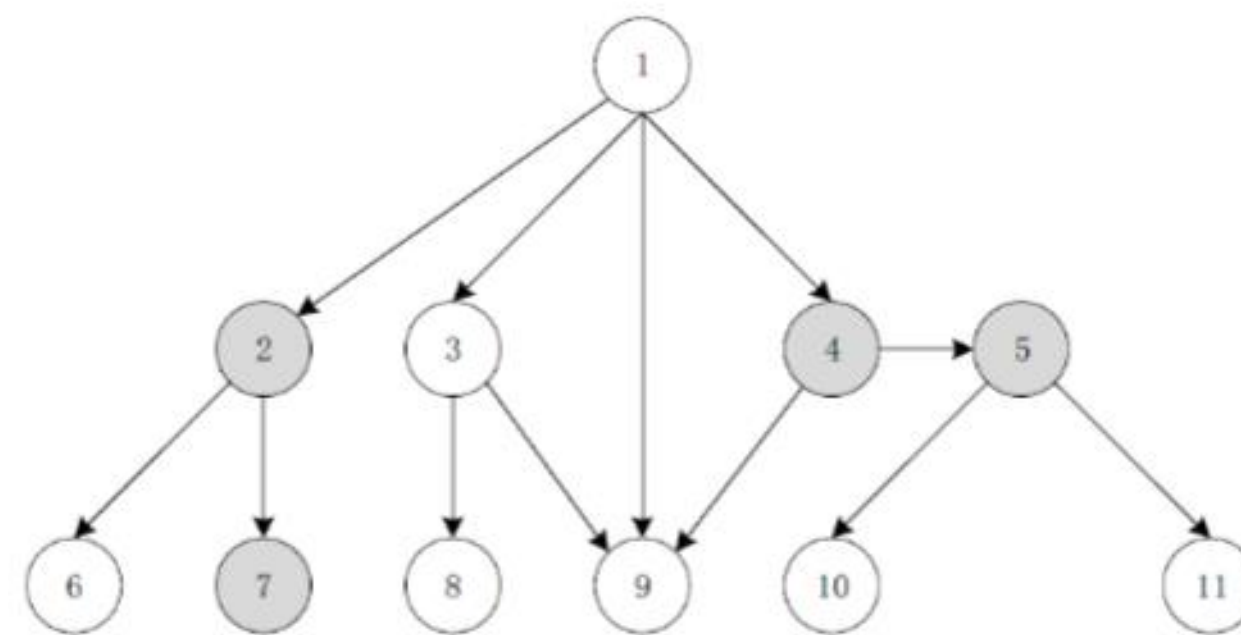


图 6.5: 成对集成示例

相邻集成

什么是相邻集成？

相邻集成是指以程序调用图的某个节点为中心，将所有调用该节点的上层节点和所有被该节点调用的下层节点放在一起进行测试。

相邻集成示例

该图表示一个程序调用图，其中阴影部分表示相邻集成过程中的一个中间结果，该中间结果包含了节点 2 的邻居节点 1、6、7 和节点 5 的邻居节点 4、10、11。

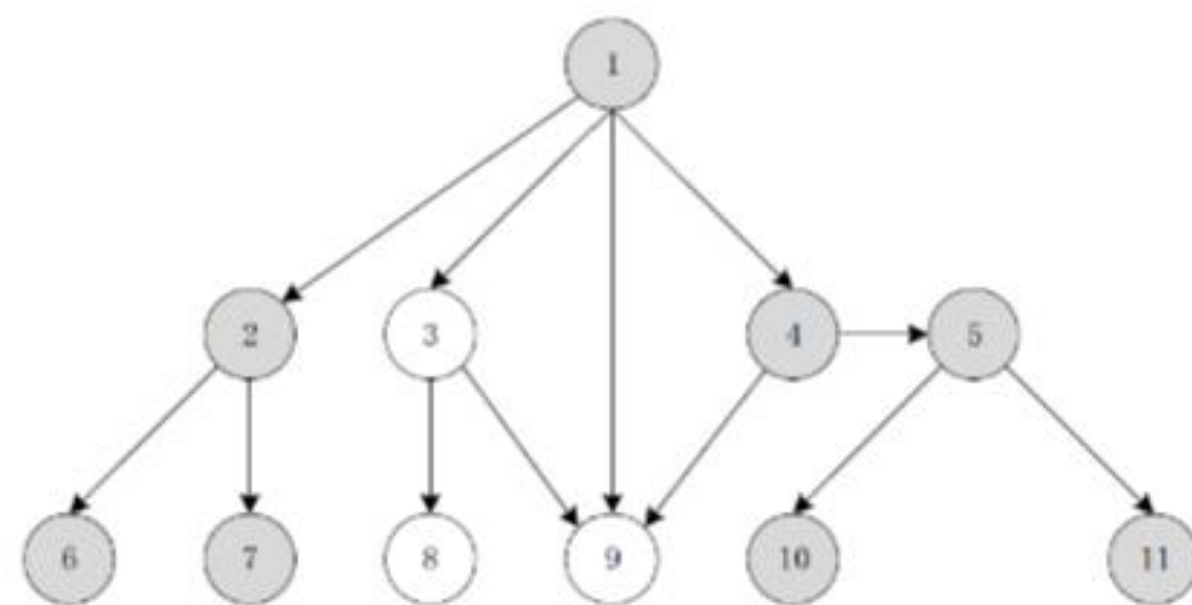


图 6.6: 相邻集成示例